What Should Loop? Mixer-Only Recurrence in Language Models

Anonymous submission

Abstract

Looped language models improve reasoning and knowledge manipulation by applying shared computation repeatedly. Existing systems usually repeat an entire layer stack, although a mixer and a dense feed-forward network (FFN) perform different operations and have different costs. We ask a narrower question: *what should loop?* We view recurrence as repeated composition of a state update and argue that an application is valuable when it exposes a new cross-position influence direction that remains observable at the task readout. Iterative Transport Rank (ITR) describes the cumulative influence trajectory; marginal ITR describes the nonredundant influence contributed by successive applications. This view motivates MixerLoop, which repeats each Gated DeltaNet mixer while applying its dense FFN once. We compare MixerLoop with no recurrence and full-stack recurrence at 15M and 110M parameters under matched data and training budgets. A finite intervention restores contextual mixer passes one at a time from a token-isolated counterfactual and measures their effect on the final next-token distribution. MixerLoop improves CORE at both scales, outperforming full recurrence at 15M and retaining 42% of its gain at 110M while using 54% of its recurrent-block projection FLOPs. Later mixer passes account for 47–54% of its finite output change and increase the true-token probability at both scales. The benefits of recurrent depth can therefore be retained without repeatedly executing the dense FFN.

Introduction

Weight-shared recurrence has become an increasingly effective way to scale the computational depth of language models without increasing their number of unique parameters. Looped Transformers, from Universal Transformers to recent models such as Ouro and LT2, repeatedly apply the same layers before producing the final token prediction (Dehghani et al. 2019; Zhu et al. 2026; Deng et al. 2026). This gives a fixed set of parameters multiple rounds of latent computation. Most systems, however, define recurrence at the block or stack level: the token mixer and feed-forward network are repeated together. Their success establishes that additional latent computation is useful, but does not reveal which operator produces the gain or how recurrent compute should be distributed within a block.

SGT provides an important operator-level result: selected softmax attention heads can be looped while the FFN is applied once (Chen et al. 2026b). Yet its selection criterion is the entropy of a one-step attention distribution. This criterion does not directly extend to a stateful linear mixer, where the same parameters can produce a different memory update at every recurrent application. LT2 shows that Gated DeltaNet (GDN) and other efficient mixers benefit from full-stack looping, but leaves open why repeated mixing helps and whether the accompanying dense FFNs must also be repeated.

We organize this question as a short chain of requirements. A loop is repeated composition of a state update. An additional composition matters only if it exposes an influence not already expressed by earlier compositions. That influence must survive the remaining network and change the task readout. Recurrence should stop when the marginal effect becomes redundant or unobservable, and additional compute should be allocated to the operator that produces the most observable effect per unit cost. This distinction is important: hidden states can move in many directions without changing the model's prediction, and a large sensitivity need not improve the correct answer.

We instantiate this view in MixerLoop, shown in Figure 1. Within each physical layer, the GDN mixer is applied four times and the dense FFN once. We compare it with a standard non-recurrent GDN and LT2-style full-stack recurrence using the same dimensions, tokenizer, corpus, optimizer, and 52.4B-token training budget. We then introduce a finite, causal readout version of Iterative Transport Rank (ITR). For one layer at a time, we remove only cross-token mixer computation, restore recurrent passes in execution order, and observe the resulting trajectory in the final predictive distribution. ITR measures the diversity accumulated by this trajectory; marginal ITR measures what each pass adds beyond previous passes. Prediction distance and the ground-truth log-probability change separately establish that the new direction has non-negligible magnitude and task value.

The results support a selective allocation of recurrent compute. MixerLoop improves the aggregate CORE score over the non-recurrent model at both 15M and 110M parameters. It dominates full recurrence at 15M; at 110M, the full model

obtains the highest raw score, while MixerLoop remains on the capability–compute frontier with 46% less recurrent-block projection compute. At both scales, passes two through four create nonredundant changes in the final distribution and jointly increase the log probability of the observed next token. A randomly initialized control produces neither a comparable readout trajectory nor a positive task contribution. Together, the architecture and intervention show that repeated dense transformation is not required for useful recurrent transport.

Related Work

Looped language models. Universal Transformers introduced depth-wise recurrence with shared parameters (Dehghani et al. 2019). Subsequent analyses connect looping to iterative algorithms and latent thoughts (Saunshi et al. 2025). Modern recurrent-depth language models bring the same idea to pretraining. Huginn increases latent computation at test time, while Ouro scales a recurrent backbone to large data regimes and finds that its gains are concentrated in knowledge manipulation rather than raw storage (Geiping et al. 2025; Zhu et al. 2026). LT2 replaces quadratic attention with linear and sparse mixers, showing that looping can iteratively refine a compact memory or expand a sparse receptive field (Deng et al. 2026). MELT reduces the memory cost of recurrent decoding (Vendrell et al. 2026). These studies establish recurrent depth as a useful scaling axis; we study the smaller allocation decision inside that recurrent computation.

Selective recurrence. SGT is the closest precedent for placing the recurrent boundary below the Transformer block. It loops selected high-entropy attention heads and applies the FFN once, substantially reducing additional training compute (Chen et al. 2026b). LoopMoE provides a complementary observation: repeated FFN computation can remain useful when successive passes activate different experts (Chen et al. 2026a). The shared lesson is not that one operator should always loop, but that useful recurrence must expose computation that changes across applications. Our focus is narrower and causal: whether successive mixer applications make distinct, finite changes to the final prediction.

Linear mixers as recurrent memory. Linear attention can be interpreted as a fast-weight memory that writes key–value associations into a compact recurrent state (Schlag, Irie, and Schmidhuber 2021). Delta-rule updates revise an existing association before writing a new one, and GDN adds input-dependent forgetting to control memory over long sequences (Yang, Kautz, and Hatamizadeh 2025). A stateful mixer is therefore a natural test case for operator-selective recurrence: applying the same mixer again can change how information is erased, written, and retrieved even though its parameters are shared. MixerLoop asks whether this refinement is sufficient to preserve useful recurrent depth without repeating the dense FFN.

Method

Allocating Recurrent Compute

Let one physical layer contain a residual GDN mixer and a residual dense FFN:

$$\mathcal{A}_i(h) = h + \mathrm{GDN}_i(\mathrm{Norm}_{A,i}(h)), \tag{1}$$

$$\mathcal{F}_i(h) = h + \mathrm{FFN}_i(\mathrm{Norm}_{F,i}(h)), \tag{2}$$

and let $\mathcal{B}_i = \mathcal{F}_i \circ \mathcal{A}_i$. A non-recurrent GDN executes

$$h^{\mathrm{NoLoop}} = (\mathcal{B}_L \circ \cdots \circ \mathcal{B}_1)(h^0). \tag{3}$$

Our FullLoop baseline follows LT2 and repeats the complete physical stack:

$$h^{\mathrm{FullLoop}} = (\mathcal{B}_L \circ \cdots \circ \mathcal{B}_1)^T(h^0). \tag{4}$$

MixerLoop instead places recurrence inside each layer:

$$h^{\mathrm{MixerLoop}} = (\mathcal{F}_L \circ \mathcal{A}_L^T) \circ \cdots \circ (\mathcal{F}_1 \circ \mathcal{A}_1^T)(h^0). \tag{5}$$

Mixer parameters are shared across the T applications, while physical layers retain distinct parameters. The three models therefore have effectively equal unique parameter counts but execute $(1,1)$, $(T,1)$, and (T,T) mixer–FFN applications per physical layer.

This boundary is motivated by function, not by a claim that FFNs are generally unhelpful under recurrence. A dense FFN is pointwise and has no direct cross-token path when repeated in isolation. In FullLoop it can still alter the input seen by a mixer on the next global pass, and sparse recurrent FFNs can activate different experts. GDN, by contrast, directly updates and reads a sequence state on every application. MixerLoop tests whether repeated stateful mixing alone provides a useful recurrent trajectory.

Let C_A and C_F denote the projection FLOPs of one mixer and FFN. FullLoop costs $T(C_A + C_F)$ per physical layer, while MixerLoop costs $TC_A + C_F$. The dense FFN accounts for 61.2–61.3% of these projections in both model sizes. With $T = 4$,

$$\frac{TC_A + C_F}{T(C_A + C_F)} = 0.541, \tag{6}$$

so MixerLoop removes 45.9% of the recurrent-block projection compute.

Finite Causal ITR

The quantity of interest is not whether hidden states change, but whether a later mixer application causes a new contextual effect that reaches the final prediction. We construct a finite counterfactual for each physical layer ℓ. The native mixer processes a sequence jointly and can use its causal convolution and recurrent GDN state. Its *context-off* counterpart applies the same mixer independently to every token, resetting both forms of cross-token state between tokens. This preserves the mixer's projections, gates, normalization, residual path, and token-local nonlinear computation; only cross-position computation is removed.

For a layer with T mixer occurrences, policy $r \in \{0, \ldots, T\}$ keeps the first r occurrences native and replaces the remaining $T - r$ with the context-off mixer. Every other layer, FFN, and downstream operation remains native.

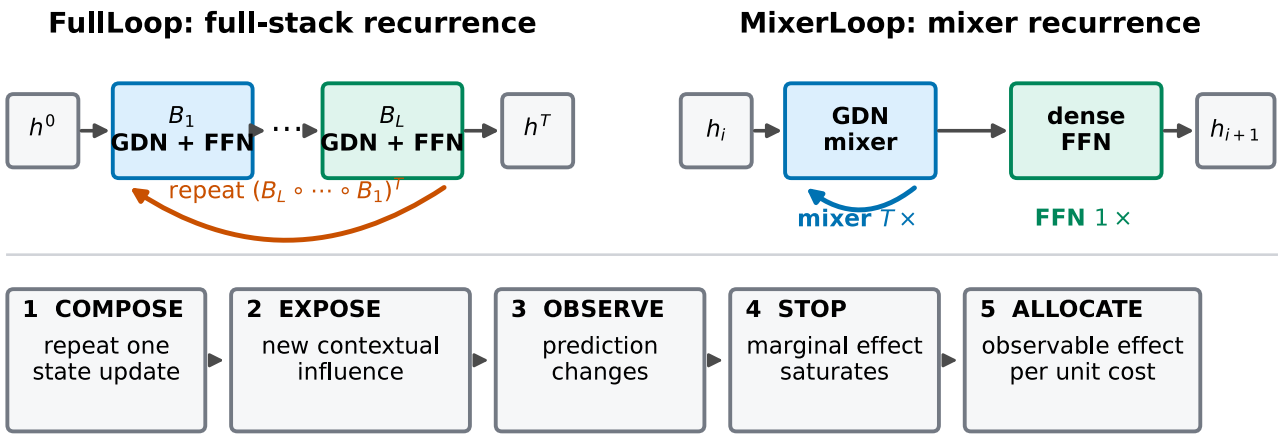

Figure 1: The recurrent boundary and the paper's organizing principle. FullLoop repeats the complete GDN–FFN stack, whereas MixerLoop repeats the mixer within each physical layer and executes the dense FFN once. Useful recurrence must create a new contextual effect, preserve it to the prediction, and justify its marginal cost.

In MixerLoop these occurrences are consecutive within a layer; in FullLoop they occur across global stack passes. Let $p_{\ell,r}(\cdot \mid x_{\le t})$ be the final next-token distribution under this intervention. Policy 0 removes contextual mixing from that layer, whereas policy T is exactly the unmodified model.

We work in the square-root probability representation

$$s_{\ell,r}(x,t) = \sqrt{p_{\ell,r}(\cdot \mid x_{\le t})}. \tag{7}$$

This choice places the trajectory in a fixed, task-level coordinate system and makes Euclidean distance equal to Hellinger distance. Concatenating vocabulary coordinates over evaluated layers, windows, and prediction positions, define

$$a_r = \mathrm{vec}_{\ell,x,t}\left[s_{\ell,r}(x,t) - s_{\ell,0}(x,t)\right], \qquad r = 1,\dots,T. \tag{8}$$

Because the only altered computation is contextual mixing in layer ℓ, a_r is the finite cross-token effect observable at the language-model readout, rather than a hidden-state sensitivity proxy.

Normalize each nonzero a_r and form the Gram matrix $K_{rs} = \langle a_r/\|a_r\|, a_s/\|a_s\|\rangle$. We define

$$\mathrm{ITR}_T = \frac{\mathrm{tr}(K)^2}{\mathrm{tr}(K^2)}. \tag{9}$$

ITR is the participation-ratio rank of the cumulative readout trajectory. It lies between one and T: one means successive policies move the prediction in the same cumulative direction, while a larger value means recurrent depth exposes a more diverse set of observable contextual effects.

For the contribution of one additional pass, let $d_r = a_r - a_{r-1}$ with $a_0 = 0$, and let Π_{r-1} project onto $\mathrm{span}\{d_1,\dots,d_{r-1}\}$. Marginal ITR is

$$\mathrm{mITR}_r = \frac{\|d_r - \Pi_{r-1} d_r\|^2}{\|d_r\|^2}, \tag{10}$$

with $\mathrm{mITR}_1 = 1$. It measures the fraction of the current finite readout update not represented by earlier updates.

Direction alone is insufficient. We therefore report the squared Hellinger effect

$$H_r^2 = \tfrac{1}{2}\|s_{\ell,r} - s_{\ell,r-1}\|^2 \tag{11}$$

and the signed ground-truth contribution

$$g_r = \log p_{\ell,r}(y \mid x_{\le t}) - \log p_{\ell,r-1}(y \mid x_{\le t}). \tag{12}$$

ITR and mITR ask whether the direction is new; H_r^2 asks whether it is large enough to alter the prediction; g_r asks whether it helps the observed target. None of these quantities substitutes for downstream evaluation.

Experiments

Setup

We evaluate NoLoop, MixerLoop, and FullLoop at two scales. The nominal 15M models contain 15.7M unique parameters, six layers, width 288, four GDN heads, and FFN width 768. The nominal 110M models contain 116.9M parameters, twelve layers, width 768, twelve heads, and FFN width 2,048. Recurrent models use $T = 4$. All models use a 32K SentencePiece tokenizer and context length 1,024. They are trained on the same 170-shard, 10.69B-token subset of ClimbMix (Diao et al. 2025), streamed for 100,000 updates and 52.43B processed tokens. The global batch contains 512 sequences. We use AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, peak learning rate 5×10^{-4}, 1,000 warmup updates, cosine decay, weight decay 0.1, and gradient clipping at 1.0. All architectures use the same fixed training seed, 1,337; we did not conduct an architecture-specific hyperparameter sweep, and fix $T = 4$ to match the released LT2 baseline rather than selecting it on the reported evaluations. Training and evaluation use two 24 GB RTX 3090 Ti GPUs on Ubuntu 22.04 with Python 3.10, PyTorch 2.13.0/CUDA 13.0, Transformers 4.57.3, and Flash Linear Attention 0.5.1.

Downstream quality is measured with the fixed 22-task CORE bundle derived from the DataComp-LM evaluation (Li et al. 2025). Each score is adjusted for its random baseline,

Model	Hella-ZS	Jeop.	WikiQA	ARC-E	ARC-C	COPA	CSQA	PIQA	OBQA	LAMB.	BoolQ	CORE
15M parameters / 52.4B training tokens												
NoLoop	4.2	0.1	**8.4**	18.0	-3.1	-16.0	1.2	20.1	3.7	13.0	-26.4	5.0
MixerLoop	4.1	**0.2**	1.6	16.7	-4.0	-16.0	**2.1**	19.5	**5.9**	**15.0**	**-6.9**	**6.5**
FullLoop	**4.6**	0.0	6.4	**19.1**	**-2.6**	**-8.0**	-0.1	**20.8**	4.8	14.7	-24.0	5.5
110M parameters / 52.4B training tokens												
NoLoop	19.6	1.3	31.5	39.1	3.4	2.0	0.7	37.6	10.7	30.6	-19.7	14.2
MixerLoop	21.1	2.4	35.3	41.9	6.9	8.0	10.4	37.8	11.7	30.7	**-12.1**	15.6
FullLoop	**24.5**	**2.8**	**38.1**	**45.1**	**7.7**	**12.0**	**14.2**	**38.3**	**14.7**	**32.4**	-21.6	**17.5**

Model	Hella.	Wino.	WinoG.	Dyck	LSAT-AR	CS-Alg.	Oper.	Repeat	SQuAD	CoQA	Lang-ID	Manip.
15M parameters / 52.4B training tokens												
NoLoop	3.4	0.4	-1.8	1.1	**7.1**	**39.5**	**13.8**	0.0	1.0	4.4	18.0	7.9
MixerLoop	3.8	**9.9**	**4.8**	**8.9**	5.4	37.0	11.0	0.0	1.3	5.5	17.6	**9.6**
FullLoop	**4.2**	7.0	-3.6	0.7	4.3	38.5	9.0	0.0	**1.7**	**5.6**	**18.4**	7.8
110M parameters / 52.4B training tokens												
NoLoop	18.9	12.1	4.2	15.0	**8.7**	**45.1**	14.8	0.0	6.5	11.6	18.0	14.1
MixerLoop	20.6	16.5	2.8	9.8	3.8	43.0	12.4	0.0	8.0	12.6	**18.6**	13.5
FullLoop	**24.3**	**18.7**	**4.5**	**19.9**	7.1	43.2	**15.2**	**3.1**	**9.8**	**14.2**	17.4	**16.1**

Table 1: CORE performance at matched parameter and training-token budgets. The upper and lower panels retain the fixed task split used in our evaluation report. Scores are chance-adjusted and multiplied by 100. CORE averages all 22 tasks; Manip. averages the 11 tasks in the lower panel. Best results within each scale are bold and second-best results are underlined.

then multiplied by 100; CORE is the unweighted mean over all tasks. We shuffle each task with seed 1,337 and sample few-shot demonstrations with seed $1{,}234 + i$ for example i. Following the motivation from Ouro, we also show the mean of the second, manipulation-heavy half of the suite rather than hiding task-level variation behind one aggregate. These are single-checkpoint comparisons; we report all task scores rather than treating examples as independent training runs.

For ITR, we sample held-out ClimbMix windows with seed 2,027 and evaluate positions in the second half of each window. The 15M analysis uses 16 windows, 16 positions per window, and all six layers; the 110M analysis uses eight windows, eight positions, and all twelve layers. Gram matrices concatenate all evaluated vocabulary directions with equal layer and window weighting. The native-prefix-T policy reproduces the unhooked model exactly (maximum absolute error 0 in all runs).

Capability per Recurrent Compute

Table 1 gives the complete capability comparison. At 15M, MixerLoop reaches 6.52 CORE, compared with 5.01 for NoLoop and 5.52 for FullLoop. It also obtains the highest manipulation aggregate, 9.57. The full-stack model is therefore dominated at this scale: it executes every FFN four times yet scores below MixerLoop. At 110M, MixerLoop improves CORE from 14.16 to 15.56, while FullLoop reaches 17.52. Relative to NoLoop, MixerLoop retains 41.5% of the full-loop improvement with the 54.1% recurrent-block compute ratio in Equation 6. The held-out NLL follows the same ordering: NoLoop, MixerLoop, and FullLoop obtain 2.995, 2.946, and 2.936 at 15M, and 2.401, 2.377, and 2.342 at 110M.

The theoretical saving also appears in wall-clock execution. We measure BF16 prefill at context 1,024 on one RTX 3090 Ti, returning only the final-token logits. At batch one, kernel-launch overhead limits the MixerLoop speedup over FullLoop to $1.12\times$ at 15M and $1.11\times$ at 110M. At the largest measured batch (32 for 15M and 16 for 110M), median latency falls from 62.1 to 46.4 ms and from 236.3 to 155.5 ms, respectively: $1.34\times$ and $1.52\times$ higher throughput. These values use six warmup forwards and 20 timed forwards. The gap between the FLOP ratio and small-batch latency is expected for several short, fused recurrent kernels; the saving becomes more visible as arithmetic utilization increases.

Do Later Mixer Passes Reach the Prediction?

Figure 2 separates novelty, magnitude, and task contribution. At 15M, the cumulative ITR of MixerLoop grows from 1 to 1.301 across four passes, nearly identical to FullLoop's 1.305. Its marginal ITR for passes two through four is 0.988, 0.871, and 0.777. These passes are not small geometric artifacts: they account for 54.2% of the total squared Hellinger change, and every pass increases the observed next-token log probability. The later three passes contribute $+0.321$ nats in aggregate, compared with $+0.210$ for FullLoop.

The same pattern survives scale. At 110M, MixerLoop reaches ITR 1.177, with marginal ITR 0.786, 0.688, and 0.672 after the first pass. Later passes account for 47.2% of its finite prediction change and add $+0.070$ nats to the true-token log probability. FullLoop follows a more diverse cumulative trajectory at this scale (ITR 1.293), but its later signed gain is slightly smaller at $+0.056$ nats. Thus higher trajectory rank is not itself a quality score: MixerLoop uses fewer operator applications while retaining a sequence of distinct, beneficial prediction corrections.

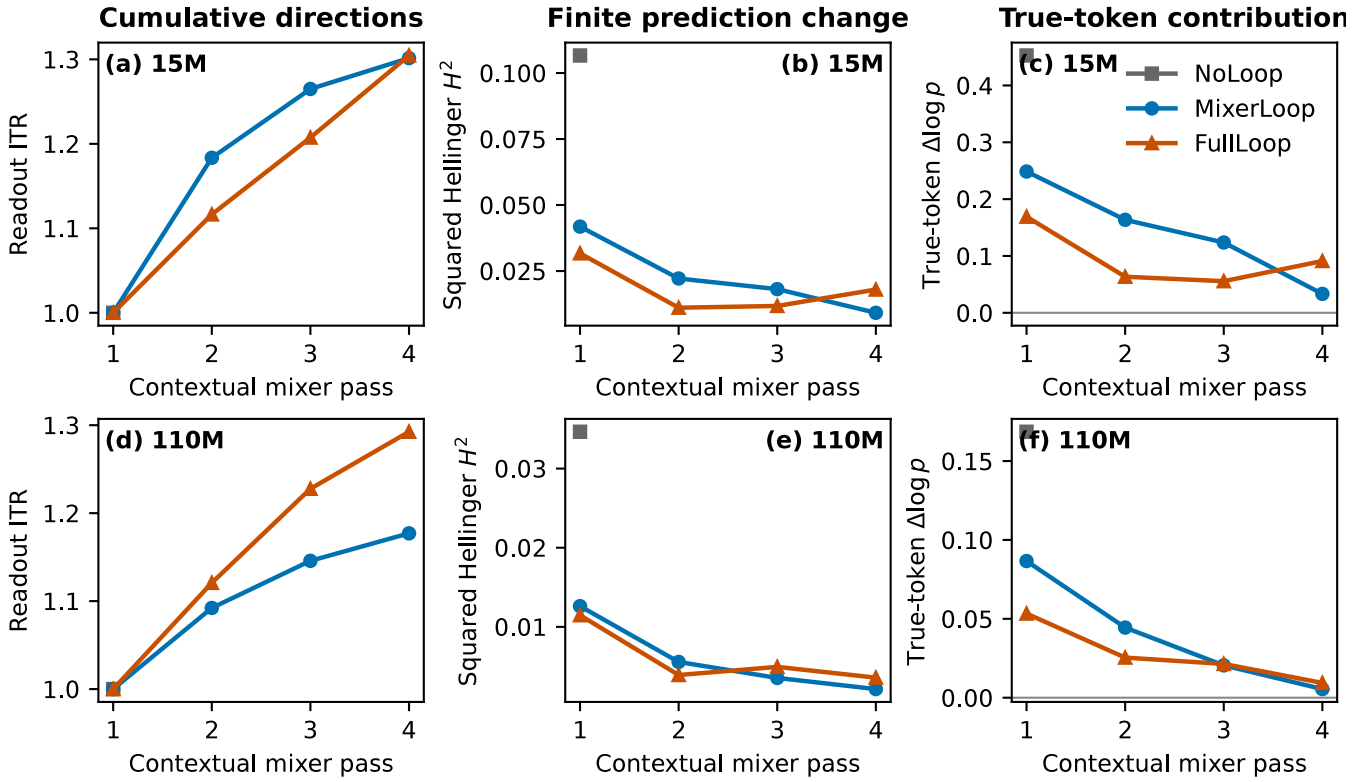

Figure 2: Finite causal readout trajectories at 15M (top) and 110M (bottom). For one layer at a time, contextual mixer passes are restored in execution order from the token-isolated policy. Left: cumulative readout ITR. Middle: squared Hellinger distance caused by each restored pass. Right: change in the ground-truth next-token log probability. Curves aggregate all evaluated layers, held-out windows, and prediction positions. NoLoop has only one mixer occurrence and therefore no later-pass trajectory.

Controls and robustness. A randomly initialized 15M MixerLoop model provides the crucial dimensionality control. Its ITR is only 1.068 and marginal ITR falls from 0.440 to 0.283 and 0.241. More importantly, each pass changes the output by only 6.4×10^{-6} squared Hellinger distance, and the later passes contribute -1.5×10^{-4} nats to the target. Non-collinearity without magnitude or task alignment is therefore rejected by the monitor. On a second set of held-out windows, the trained 15M ITR is 1.313 for MixerLoop and 1.284 for FullLoop; increasing the context from 128 to 256 tokens gives 1.327 and 1.311. The conclusion is stable to the sampled windows and context length.

A layer-wise allocation signal. Figure 3 asks whether aggregate novelty is concentrated in a few physical layers. At 15M, 16 of 18 later MixerLoop layer–pass pairs have marginal ITR above 0.5, and all 18 have positive true-token contribution. At 110M, the corresponding counts are 19 of 36 for novelty and 30 of 36 for positive contribution. FullLoop maintains marginal ITR above 0.5 for every later layer–pass pair at both scales, but only 27 of 36 contributions are positive at 110M. The monitor therefore distinguishes two decisions: whether an operator family is worth recurring at all, and where its marginal readout effect has begun to saturate. The latter can guide layer- or head-selective recurrence without using hidden-state movement as the selection target.

Discussion

The intervention answers a more specific question than ordinary loop truncation. It does not ask whether a model trained with four passes deteriorates when three are deleted. Instead, all computation remains in place, token-local mixer behavior is preserved, and contextual passes are restored one at a time. The measured trajectory therefore identifies finite contextual effects that survive the complete downstream network.

Two observations refine the interpretation of ITR. First, the cumulative ranks remain between 1.18 and 1.31 rather than approaching four, even when individual marginal updates are substantially nonredundant. The recurrent mixer behaves like a sequence of distinct corrections that accumulate toward a coherent predictive direction, not four independent algorithms. Second, rank must be read jointly with effect magnitude and sign. FullLoop has higher ITR at 110M, whereas MixerLoop has the larger later true-token gain; a random network can assign a large *fraction* of tiny changes to later passes while doing no useful work. This is why ITR is an allocation diagnostic rather than a stand-alone benchmark.

Our claims are bounded in three ways. We study dense GDN–FFN models with four recurrent passes at 15M and 110M parameters. SGT suggests that the same readout-level test can guide recurrence in softmax attention, but we do not claim architecture-independent rankings without that ex-

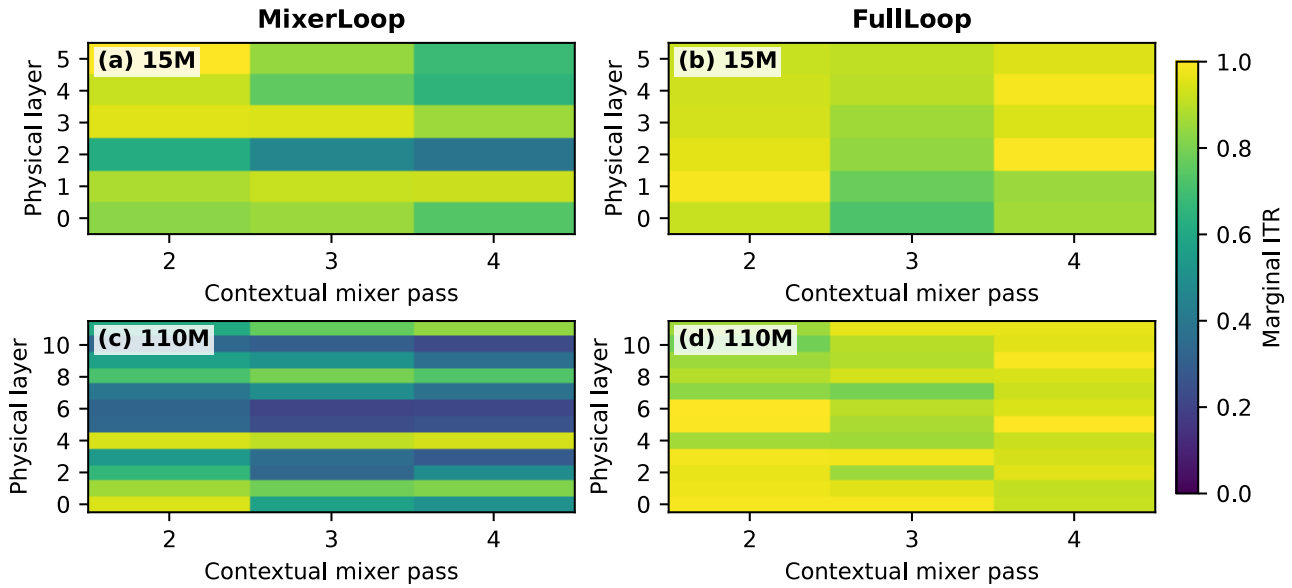

Figure 3: Layer-wise marginal ITR after the first contextual pass. Each cell shows the fraction of a pass's finite readout update outside the span of earlier updates for the same physical layer. MixerLoop retains substantial novelty in most 15M layers and a more selective subset at 110M; FullLoop maintains high novelty across the stack. Layers are indexed from bottom to top.

periment. The context-off policy is an ordered, prefix attribution; interactions can make a different restoration order yield different marginal values. Finally, each architecture is represented by one training run. The full task table and finite-intervention controls support the reported models, while training-seed uncertainty remains to be measured at larger compute budgets.

Conclusion

Looped language models need not repeat every expensive operator. MixerLoop applies recurrent depth to the stateful token mixer and executes the dense FFN once, improving CORE over a non-recurrent GDN at both evaluated scales while using 54% of FullLoop's recurrent-block projection compute. Finite causal ITR shows why this allocation is viable: later mixer applications create new contextual directions that change the final distribution and improve the observed next token. The broader design rule is simple. Allocate recurrent compute where successive applications continue to produce nonredundant, task-observable effects; stop when either novelty or effect vanishes.

References

Chen, W.; Li, T.; Huang, W.; Yin, Y.; Shang, L.; and Qin, C. 2026a. LoopMoE: Unifying Iterative Computation with Mixture-of-Experts for Language Modeling. arXiv:2606.04438.

Chen, Y.; Chen, Y.; Yang, Y.; et al. 2026b. Sparse Growing Transformer: Training-Time Sparse Depth Allocation via Progressive Attention Looping. In *Findings of the Association for Computational Linguistics: ACL 2026*, 6168–6193. San Diego, California, United States: Association for Computational Linguistics.

Dehghani, M.; Gouws, S.; Vinyals, O.; Uszkoreit, J.; et al. 2019. Universal Transformers. arXiv:1807.03819.

Deng, C.; Zhang, Y.; Zhu, R.-J.; Xu, Y.; et al. 2026. LT2: Linear-Time Looped Transformers. arXiv:2605.20670.

Diao, S.; Yang, Y.; Fu, Y.; et al. 2025. Nemotron-CLIMB: Clustering-Based Iterative Data Mixture Bootstrapping for Language Model Pre-Training. arXiv:2504.13161.

Geiping, J.; McLeish, S.; Jain, N.; Kirchenbauer, J.; et al. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. arXiv:2502.05171.

Li, J.; Fang, A.; Smyrnis, G.; Ivgi, M.; et al. 2025. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. arXiv:2406.11794.

Saunshi, N.; Dikkala, N.; Li, Z.; Kumar, S.; et al. 2025. Reasoning with Latent Thoughts: On the Power of Looped Transformers. arXiv:2502.17416.

Schlag, I.; Irie, K.; and Schmidhuber, J. 2021. Linear Transformers Are Secretly Fast Weight Programmers. arXiv:2102.11174.

Vendrell, V. C.; Masdemont, A. P.; Grillo, N.; Ros-Giralt, J.; et al. 2026. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models. arXiv:2605.07721.

Yang, S.; Kautz, J.; and Hatamizadeh, A. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. arXiv:2412.06464.

Zhu, R.-J.; Wang, Z.; Hua, K.; Zhang, T.; et al. 2026. Scaling Latent Reasoning via Looped Language Models. arXiv:2510.25741.